

Ejercicio 1

- a) Implemente la clase `Fecha` en C++ con los métodos usuales (constructor por defecto, constructor común, constructor de copia y funciones selectoras) y además las siguientes operaciones:

```

== : Fecha x Fecha -> bool    (determina si ambas fechas son iguales)
<  : Fecha x Fecha -> bool    (determina si la primer fecha es anterior a la segunda)
++ : Fecha -> Fecha           (suma un día a una fecha dada)
+  : Fecha x int -> Fecha      (suma una cantidad dada de días a una fecha dada)
-  : Fecha x Fecha -> int      (cantidad de días de diferencia entre dos fechas dadas)
esValida: Fecha -> bool

```

Para las primeras cinco operaciones se **sobrecargarán** los operadores correspondientes de C++. En particular para el caso del operador `++` se hará en sus dos versiones (**prefija** y **postfija**). Al implementar, tenga en cuenta que la primer `Fecha` corresponde siempre al objeto implícito.

Sugerencia: implemente un método **privado** auxiliar `void diaSiguiente();` que transforme la fecha correspondiente al objeto implícito en la fecha correspondiente al día siguiente (por ejemplo, el día siguiente al 30/11/2000 es 1/12/2000). Le será de utilidad para varias de las operaciones.

- b) Haga un programa de prueba (`main`) que pruebe los métodos de la clase `Fecha`.

Ejercicio 2

- a) Implemente una clase `Estadia` en C++ que represente la estadía de un huésped en un hotel. Una estadía está formada por un número de estadía (`int`), una fecha de ingreso y una fecha de egreso. Además de los métodos usuales (constructores, selectores y modificadores), deberá brindarse otro que devuelva la cantidad de días que duró la estadía.
- b) Haga un programa de prueba (`main`) que pruebe los métodos de la clase `Estadia`.

Ejercicio 3

Dada la siguiente especificación en UML para un **iterador** de Estadías del ejercicio anterior:

<< colección >> ITERADOR
– iter: Array con tope
+ Iterador () + insertar (Estadia) : void + hayMasEstadias () : boolean + proximaEstadia () : Estadia

Implemente la clase `Iterador` en C++ usando una estructura de **Arreglo con tope** según la siguiente definición y luego haga un programa de prueba (`main`).

```

private:    Estadia iter [TAMANIO];
            int tope;
            int actual;

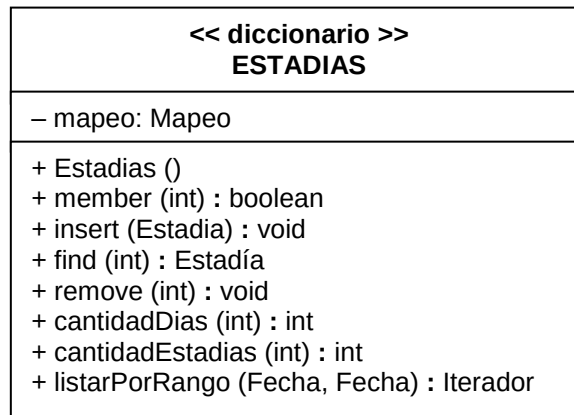
```

Notas:

- Incluya solamente constructor **por defecto** en la implementación.
- **TAMANIO** es una constante simbólica previamente definida.
- **tope** indica la cantidad de objetos almacenados en el iterador.
- **actual** indica la posición actual del iterador.
- Puede asumir que la cantidad de objetos del iterador no excederá **TAMANIO**.

Ejercicio 4

Dada la siguiente especificación en UML para un **diccionario** de Estadías del ejercicio 2, donde la clave que identifica a cada estadía es su número de estadía:



- Las primitivas **member**, **insert**, **find** y **remove** tienen el comportamiento usual para un diccionario.
- El método **cantidadDías** devuelve la cantidad de días que duró la estadía correspondiente al número ingresado por parámetro. (Precondición: La estadía con dicho número es miembro del diccionario).
- El método **cantidadEstadías** devuelve la cantidad de estadías cuya duración (en cantidad de días) coincide con el valor ingresado por parámetro.
- El método **listarPorRango** devuelve un iterador conteniendo todas las estadías contenidas en el rango de fechas determinado por los parámetros recibidos.

Implemente la clase `Estadías` en C++ usando una estructura de **Mapeo** según la siguiente definición y luego haga un programa de prueba (`main`).

```
private:    struct Celda { bool existe;
                                Estadia info;
                                };
                                Celda mapeo [TAMANIO];
```

Notas:

- Incluya solamente constructor **por defecto** en la implementación.
- **TAMANIO** es una constante simbólica previamente definida.
- Asuma que el número de cada estadía está en el rango **0..TAMANIO-1**.

Ejercicio 5

Compile y ejecute los archivos `MiClase.h`, `MiClase.cpp` y `main.cpp`. Discuta y explique paso a paso los resultados de la ejecución de dicho programa. Los archivos fuente se encuentran en la página web del curso.